

WTB Failure Modes Explained

Document Purpose: Explain how WTB (Workflow Test Bench) addresses common failure modes in AI/ML workflows, with detailed mechanisms, SDK usage, and cost analysis.

Version: 1.8 (2026-02-07)

Executive Summary

WTB addresses **7 critical failure modes** in AI/ML workflows. For each failure mode, this document explains:

1. **The Problem** - What can go wrong
 2. **The Solution** - How WTB solves it
 - **Principle** - The underlying mechanism
 - **SDK Usage** - How to use it in practice
 3. **Cost Analysis** - The overhead vs. benefit trade-off
-

Failure Mode 1: Phantom Answer (Non-Deterministic LLM Responses)

The Problem

When an LLM call fails and retries, each retry may produce a **different answer** due to:

- Temperature > 0 (stochastic sampling)
- Model updates between calls
- Different prompt formatting on retry

Impact: A clinician may receive different treatment recommendations depending on which retry succeeded.

SOTP (State-of-the-Practice):

Retry 1: "Recommend Drug A at 500mg" ← Fails, discarded

Retry 2: "Recommend Drug B at 250mg" ← Succeeds, returned

Result: Answer depends on retry timing, not clinical evidence.

Reproducibility: 35.9%

The Solution

Principle: Idempotency Keys

WTB uses **execution-scoped idempotency keys** to ensure identical responses across retries:

```
# How it works internally
class IdempotentLLMService:
    def call(self, prompt: str, execution_id: str) -> str:
        # Generate idempotency key from content + execution context
        key = hash(f"{prompt}:{execution_id}")

        # Check if we've seen this exact call before
        if key in self._cache:
            return self._cache[key] # Return identical answer

        # First call - make real API request
        response = self._llm.generate(prompt)
        self._cache[key] = response
        return response
```

The key insight: **execution_id is tied to the workflow run**, not the retry attempt. All retries within the same execution get the same answer.

SDK Usage

```
from wtb.sdk import WorkflowTestBench

# Create test bench with idempotency enabled (default)
bench = WorkflowTestBench.create(
    db_path="data/wtb.db",
    enable_idempotency=True, # Default: True
)

# Run workflow - retries automatically use same execution_id
result = bench.run_workflow(
```

```
workflow=my_workflow,
initial_state={"query": "What drug for diabetes?"},
)

# Even if internal LLM calls retry, answer is deterministic
print(result.state["answer"]) # Always same answer for same input
```

One-liner: Just use `WorkflowTestBench.create()` - idempotency is enabled by default.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
API calls (3 retries × 5 docs)	15 real calls	5 real + 10 cached	Hash computation: ~0.01ms
API cost	\$0.015	\$0.005	-
Cache storage	None	~1KB per call	SQLite write: ~0.5ms
Total API cost	2.21x baseline	1.00x baseline	~0.5ms per call

Net Benefit: 54.7% cost reduction, ~0.5ms overhead per call.

Failure Mode 2: Phantom Citation (Orphan Vectors)

The Problem

When writing to multiple stores (SQL + VectorDB), a crash between writes creates **orphan records**:

```
Transaction:
1. Write to SQL: citation_id=123, text="Study shows..." ✓
2. Write to VectorDB: vector_id=123, embedding=[...] ← CRASH

Result: SQL has citation_id=123, but VectorDB has no matching vector.
```

Citation cannot be retrieved by semantic search.
Audit trail is broken.

The Solution

Principle: Compensating Transactions (Saga Pattern)

WTB implements the **Saga pattern** with compensating transactions:

```
# How it works internally
class UnitOfWork:
    def __enter__(self):
        self._operations = []
        self._compensations = []

    def add_sql(self, record):
        self._operations.append(("sql", record))
        # Register compensation (reverse action)
        self._compensations.append(lambda: self._sql.delete(record.id))

    def add_vector(self, vector):
        self._operations.append(("vector", vector))
        self._compensations.append(lambda: self._vectordb.delete(vector.id))

    def commit(self):
        try:
            for op_type, data in self._operations:
                self._execute(op_type, data)
        except Exception:
            # Rollback in reverse order
            for compensation in reversed(self._compensations):
                compensation()
            raise
```

Key insight: Each write registers its "undo" action. On failure, WTB executes compensations in reverse order.

SDK Usage

```
from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# Use Unit of Work for multi-store operations
with bench.unit_of_work() as uow:
    # Add to SQL
```

```

uow.citations.add(Citation(id="123", text="Study shows..."))

# Add to VectorDB
uow.vectors.add(Vector(id="123", embedding=[0.1, 0.2, ...]))

# Commit - if either fails, both are rolled back
uow.commit()

# Or use the high-level API (UoW is implicit)
bench.add_citation_with_embedding(
    citation_id="123",
    text="Study shows...",
    embedding=[0.1, 0.2, ...],
) # Atomic: both succeed or both fail

```

One-liner: Wrap multi-store operations in `with bench.unit_of_work()` as `uow`.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Write operations	2 writes	2 writes + 2 compensation registrations	~0.1ms
Crash recovery	Manual cleanup	Automatic rollback	~10ms on failure
Audit integrity	45% orphan rate	0% orphan rate	-
Total overhead	N/A	~0.1ms per operation	~10ms on failure

Net Benefit: 100% data integrity, ~0.1ms overhead per operation.

Failure Mode 3: Disordered Protocol (Race Conditions)

The Problem

In concurrent execution, steps may complete out of order:

Clinical Protocol (MUST be sequential):

1. Check allergies
2. Verify drug interactions
3. Administer chemotherapy

Concurrent Execution (SOTP):

- Thread 1: Check allergies (100ms delay)
- Thread 2: Verify interactions (50ms delay)
- Thread 3: Administer chemo (10ms delay) ← Completes FIRST!

Result: Chemo administered before allergy check.

The Solution

Principle: LangGraph Checkpoint Barriers

WTB uses **LangGraph's StateGraph** with checkpoints as execution barriers:

```
# How it works internally
builder = StateGraph(ProtocolState)
builder.add_node("check_allergies", check_allergies_node)
builder.add_node("verify_interactions", verify_interactions_node)
builder.add_node("administer_chemo", administer_chemo_node)

# Sequential edges enforce ordering
builder.add_edge(START, "check_allergies")
builder.add_edge("check_allergies", "verify_interactions") # Must wait
builder.add_edge("verify_interactions", "administer_chemo") # Must wait
builder.add_edge("administer_chemo", END)

# Compile with checkpointer - each edge creates a barrier
graph = builder.compile(checkpointer=SqliteSaver.from_conn_string(db_path))
```

Key insight: Each edge in the graph is a **checkpoint barrier**. The next node cannot start until the previous node's checkpoint is persisted.

SDK Usage

```
from wtb.sdk import WorkflowTestBench, WorkflowBuilder

bench = WorkflowTestBench.create(db_path="data/wtb.db")
```

```
# Define workflow with explicit ordering
workflow = (
    WorkflowBuilder("clinical_protocol")
    .add_node("check_allergies", check_allergies_fn)
    .add_node("verify_interactions", verify_interactions_fn)
    .add_node("administer_chemo", administer_chemo_fn)
    .add_sequential_edges([ # Enforces order
        "check_allergies",
        "verify_interactions",
        "administer_chemo",
    ])
    .build()
)

# Run - ordering is guaranteed
result = bench.run_workflow(workflow, initial_state)
# check_allergies ALWAYS completes before verify_interactions
```

One-liner: Use `.add_sequential_edges()` to enforce step ordering.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Execution model	Concurrent (race)	Sequential (ordered)	-
Checkpoint write	None	1 per step	~1-2ms per step
Ordering guarantee	0%	100%	-
Total overhead	N/A	~1-2ms per step	Kendall's tau: 1.0

Net Benefit: 100% ordering guarantee, ~1-2ms overhead per step.

Failure Mode 4: Zombie Guideline (Stale Cache)

The Problem

When data is updated, cached copies become **stale and dangerous**:

Timeline:

T0: Cache guideline: "Metformin max dose: 2000mg"

T1: FDA issues safety alert: "Max dose: 1000mg for eGFR < 45"

T2: Query hits STALE cache: "Max dose: 2000mg" ← DANGEROUS

Patient with eGFR=35 receives 2000mg instead of safe 1000mg.

The Solution

Principle: Outbox Pattern for Cache Invalidation

WTB uses the **Outbox Pattern** to ensure cache invalidation is atomic with data updates:

```
# How it works internally
class GuidelineCacheWithOutbox:
    def update_guideline(self, drug: str, new_data: dict, uow: UnitOfWork):
        # 1. Update backend
        self._backend.update(drug, new_data)

        # 2. Queue invalidation event in SAME transaction
        outbox_event = OutboxEvent.create(
            event_type=OutboxEventType.CACHE_INVALIDATE,
            aggregate_id=drug,
            payload={"version": new_data["version"]},
        )
        uow.outbox.add(outbox_event)

        # 3. Invalidate cache
        self._cache.invalidate(drug)

        # 4. Atomic commit - both update and invalidation
        uow.commit()
```

Key insight: The invalidation event is written to the **outbox table** in the same transaction as the data update. Even if the process crashes after commit, the outbox processor will eventually invalidate the cache.

SDK Usage


```

from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(
    db_path="data/wtb.db",
    enable_outbox=True, # Default: True
)

# Update with automatic cache invalidation
with bench.unit_of_work() as uow:
    # Update backend data
    uow.guidelines.update("metformin", {
        "max_dose": "1000mg",
        "condition": "eGFR < 45",
        "version": "v2.1-SAFETY",
    })

    # Cache is automatically invalidated via outbox
    uow.commit()

# Or use high-level API
bench.update_guideline(
    drug="metformin",
    data={"max_dose": "1000mg", ...},
    invalidate_cache=True, # Default: True
)

```

One-liner: Use `uow.commit()` - cache invalidation is automatic via outbox.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Stale read rate	73%	20% (detected)	-
Outbox event write	None	1 per update	~0.5ms
Cache invalidation	Manual	Automatic	~0.1ms
Safety score	0.21	0.79	-
Total overhead	N/A	~0.6ms per update	3.8x safety improvement

Net Benefit: 3.8x safety improvement, ~0.6ms overhead per update.

Failure Mode 5: Cohort Duplication (Double-Counting)

The Problem

When processing batches, a crash-and-restart may **re-process already-counted items**:

```
Processing 100 patients:
  Patients 1-50: Processed, inserted to VectorDB  ✓
  Patient 51: CRASH

SOTP Restart (from beginning):
  Patients 1-50: Re-processed, RE-INSERTED  ← DUPLICATES!
  Patients 51-100: Processed, inserted

VectorDB now has 150 records (50 duplicates).
```

The Solution

Principle: Checkpoint-Based Resume with ID Tracking

WTB tracks processed IDs in checkpoint state, enabling **exactly-once processing**:

```
# How it works internally
@node
def process_batch(state: BatchState) -> BatchState:
    processed = set(state.get("processed_ids", []))

    for item in state["items"]:
        if item.id in processed:
            continue # Skip - already in checkpoint

        # Process item
        result = process(item)
        store.insert(item.id, result)

        # Track in state (checkpointed automatically)
        processed.add(item.id)

    return {"processed_ids": list(processed)}
```

Key insight: The `processed_ids` set is part of the checkpoint state. On restart, WTB loads the checkpoint and skips already-processed items.

SDK Usage

```
from wtb.sdk import WorkflowTestBench, BatchRunner

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# Use BatchRunner for exactly-once processing
runner = bench.create_batch_runner(
    checkpoint_interval=10, # Checkpoint every 10 items
)

# Run batch - automatically resumes from checkpoint on restart
result = runner.run_batch(
    items=patients,
    processor=process_patient,
)

# Or use the graph factory pattern for Ray distributed execution
from wtb.sdk import RayBatchTestRunner

ray_runner = RayBatchTestRunner.create(
    db_path="data/wtb.db",
    num_workers=4,
)

# Distributed batch with checkpoint support
result = ray_runner.run_batch_test(batch_test)
```

One-liner: Use `bench.create_batch_runner()` for exactly-once batch processing.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Duplicate rate	50% (crash at midpoint)	0%	-
Checkpoint write	None	1 per interval	~1-2ms per checkpoint
Resume time	Full restart	From checkpoint	~5ms to load checkpoint

Cost Item	SOTP	WTB	WTB Overhead
Total overhead	N/A	~1-2ms per checkpoint	100% data integrity

Net Benefit: 100% data integrity, ~1-2ms overhead per checkpoint interval.

Failure Mode 6: Diagnostic Cascade (Blast Radius)

The Problem

When a node fails in a pipeline, SOTP restarts the **entire pipeline**:

```
5-Node Pipeline:
Node 1: ✓ (10 seconds)
Node 2: ✓ (10 seconds)
Node 3: ✗ FAIL (transient error)
Node 4: Not reached
Node 5: Not reached

SOTP Retry: Re-run Node 1, 2, 3, 4, 5
            Blast radius: 100% (all nodes re-executed)
            Wasted: 20 seconds of completed work
```

The Solution

Principle: Per-Node Checkpoint with Selective Resume

WTB creates checkpoints at each node boundary, enabling **minimal re-execution**:

```
# How it works internally
def run_with_checkpoints(graph, config, initial_state):
    try:
        # Each node creates automatic checkpoint
        result = graph.invoke(initial_state, config)
    except Exception:
        # On failure, get checkpoint history
        history = graph.get_state_history(config)
        last_good = history[0] # Most recent successful checkpoint
```

```
# Resume from last good checkpoint
result = graph.invoke(
    last_good.values,
    config,
    checkpoint_id=last_good.checkpoint_id,
)

return result
```

Key insight: LangGraph's `invoke()` creates a checkpoint after each node. On failure, WTB resumes from the last successful checkpoint, skipping already-completed nodes.

SDK Usage

```
from wtb.sdk import WorkflowTestBench

bench = WorkflowTestBench.create(db_path="data/wtb.db")

# Run workflow with automatic checkpoint resume
result = bench.run_workflow(
    workflow=my_workflow,
    initial_state=initial_state,
    retry_from_checkpoint=True, # Default: True
)

# Or manually control rollback
execution = bench.run_workflow(workflow, initial_state)

# Later: rollback to specific checkpoint
rolled_back = bench.rollback(
    execution_id=execution.id,
    checkpoint_id="checkpoint_abc123",
)

# Or fork from checkpoint for exploration
forked = bench.fork(
    execution_id=execution.id,
    checkpoint_id="checkpoint_abc123",
    new_state={"exploration_mode": True},
)
```

One-liner: Set `retry_from_checkpoint=True` (default) for automatic minimal re-execution.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Blast radius (fail at Node 3/5)	100%	20%	-
Re-execution cost	5x node cost	1x node cost	-
Checkpoint write	None	1 per node	~1-2ms per node
Rollback time	Full restart	Load checkpoint	~5ms
Total overhead	N/A	~1-2ms per node	80% re-execution savings

Net Benefit: 80% re-execution savings, ~1-2ms overhead per node.

Failure Mode 7: Trial Contamination (Workspace Leakage)

The Problem

In A/B testing, shared file systems allow data leakage between variants:

A/B Test:

Variant A (Treatment): Writes to /shared/results.json

Variant B (Control): Reads /shared/results.json ← CONTAMINATED!

The control sees treatment data, invalidating the experiment.

The Solution

Principle: Per-Variant Workspace Isolation

WTB creates **isolated workspaces** for each variant with file tracking:

```
# How it works internally
class WorkspaceManager:
    def create_workspace(self, batch_id: str, variant: str) -> Workspace:
        # Create isolated directory
        path = self._base / f"batch_{batch_id}" / variant
        path.mkdir(parents=True, exist_ok=True)

        return Workspace(
            workspace_id=f"ws-{uuid4()}",
            root_path=path,
            file_tracker=SqliteFileTrackingService(path),
        )

    def cleanup_workspace(self, workspace: Workspace, preserve: bool = False):
        if not preserve:
            shutil.rmtree(workspace.root_path)
```

Key insight: Each variant runs in its own directory. File tracking ensures no cross-contamination and enables rollback of file changes.

SDK Usage

```
from wtb.sdk import RayBatchTestRunner, VariantCombination

# Create runner with workspace isolation
runner = RayBatchTestRunner.create(
    db_path="data/wtb.db",
    workspace_config={
        "base_path": "workspaces/",
        "isolate_variants": True, # Default: True
    },
)

# Define variants
batch_test = BatchTest(
    workflow_id="ab_test",
    variant_combinations=[
        VariantCombination(name="Treatment_A", variants={"model": "gpt-4"}),
        VariantCombination(name="Control_B", variants={"model": "gpt-3.5"}),
    ],
)

# Run - each variant gets isolated workspace
result = runner.run_batch_test(batch_test)

# Workspaces:
#   workspaces/batch_xxx/Treatment_A/  ← Isolated
#   workspaces/batch_xxx/Control_B/    ← Isolated
```

One-liner: Set `isolate_variants=True` (default) for automatic workspace isolation.

Cost Analysis

Cost Item	SOTP	WTB	WTB Overhead
Contamination rate	100%	0%	-
Workspace creation	None	1 per variant	~10ms
File tracking	None	Per file write	~0.5ms per file
Cleanup	Manual	Automatic	~50ms per workspace
Total overhead	N/A	~60ms per variant	100% isolation

Net Benefit: 100% experiment isolation, ~60ms overhead per variant.

Rollback and Fork Operations

Principle: Graph Factory Pattern

For distributed execution (Ray), WTB uses **importable graph factory references** instead of serializing graphs:

```
# How it works internally
@dataclass
class VariantCombination:
    name: str
    variants: Dict[str, str]
    # Serializable reference to graph factory
    graph_factory_module: Optional[str] = None # "myapp.workflows"
    graph_factory_name: Optional[str] = None # "create_graph"

    def create_graph(self) -> CompiledStateGraph:
        module = importlib.import_module(self.graph_factory_module)
        factory = getattr(module, self.graph_factory_name)
        return factory()
```

SDK Usage for Rollback/Fork


```
from wtb.sdk import RayBatchTestRunner

runner = RayBatchTestRunner.create(db_path="data/wtb.db")

# Run batch test
result = runner.run_batch_test(batch_test)

# Create coordinator for rollback/fork
coordinator = runner.create_rollback_coordinator()

# Rollback to checkpoint (requires graph for LangGraph operations)
from myapp.workflows import create_my_graph

execution = coordinator.rollback(
    execution_id="exec-123",
    checkpoint_id="cp-456",
    graph=create_my_graph(), # Required for LangGraph state adapter
)

# Fork from checkpoint (non-destructive)
forked = coordinator.fork(
    execution_id="exec-123",
    checkpoint_id="cp-456",
    new_state={"exploration_mode": True},
    graph=create_my_graph(),
)
```

Cost Analysis for Rollback/Fork

Operation	Overhead	Components
Graph factory import	~10ms	<code>importlib.import_module()</code>
Session initialization	~2ms	Connect to execution's checkpoint history
Outbox event write	~0.5ms	ACID audit trail
Checkpoint load	~5ms	SQLite read
Total rollback	~20ms	Full state restoration
Total fork	~25ms	New execution + checkpoint copy

Overall Cost Summary

Per-Solution Overhead

Failure Mode	Solution	Overhead	Benefit
Phantom Answer	Idempotency Keys	~0.5ms/call	54.7% API cost reduction
Phantom Citation	Compensating Transactions	~0.1ms/op, ~10ms on failure	100% data integrity
Disordered Protocol	Checkpoint Barriers	~1-2ms/step	100% ordering guarantee
Zombie Guideline	Outbox Invalidation	~0.6ms/update	3.8x safety improvement
Cohort Duplication	Checkpoint Resume	~1-2ms/checkpoint	100% data integrity
Diagnostic Cascade	Per-Node Checkpoint	~1-2ms/node	80% re-execution savings
Trial Contamination	Workspace Isolation	~60ms/variant	100% experiment isolation

When to Use WTB

Scenario	Recommended	Reason
LLM-heavy pipelines	Yes	54-67% API cost savings
Clinical/Research workflows	Yes	Reproducibility, audit trail
Long-running jobs	Yes	Checkpoint resume, minimal re-execution
Compliance requirements	Yes	ACID transactions, outbox audit
Simple single-step tasks	No	Overhead not justified
Sub-millisecond latency required	No	~1-2ms checkpoint overhead

Quick Start

```
from wtb.sdk import WorkflowTestBench

# One line to get all protections
bench = WorkflowTestBench.create(db_path="data/wtb.db")

# Run workflow with full ACID compliance
result = bench.run_workflow(my_workflow, initial_state)
```